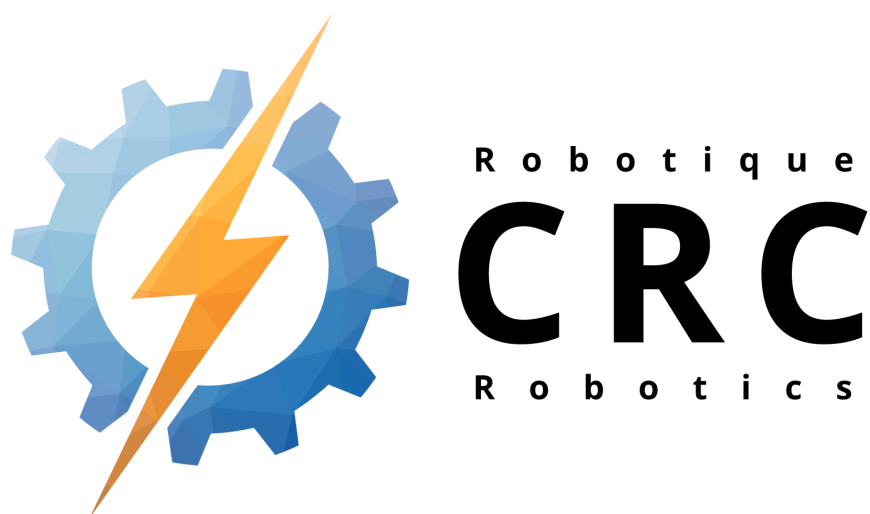




CRC
**PROGRAMMING
COMPETITION**



**PROBLEM BOOKLET
BLOCK A**

A FEW NOTES

- The complete rules are in section 4 of the rulebook. **Don't forget about multipliers!**
- You have until **20h29** to submit your solutions. Please submit the full folder in the same format as it was given to you with your solutions in the .py files.
- Feel free to ask organizers questions and discuss the problem with your team members. It is not an exam!
- **We are giving you quick and easy-to-use template files. You have to use them! We are also using templates to correct your solutions!**

USING THE TEMPLATE FILE

- All the files contain a function called `solve()` where you have to put your code. **DO NOT CHANGE THE NAME OF THAT FUNCTION!**
- To run the tests you will have to open a terminal (we strongly recommend to use `vscode`) and run the command `pytest`. If the command is ran in the folder containing all the problems, all tests are going to be ran
- To run a specific test, run it in the folder containing the test you want. You can do the same for sections, just navigate in the folder before executing the command `pytest`

STRUCTURE

Every problem contains a small introduction like this about the basics of the problem and what is required to solve it. Points distribution is also given here for the problems.

Input and output specification:

In these two sections, we specify what the inputs will be and what form they will take, and we also say what outputs are required for the code to produce and in what format they shall be.

Sample input and output:

In these two sections, you will find a sample input (that often has multiple entries itself) in the sample input and what your program should produce for such an input in the sample output.

First output explanation:

Sometimes, the problem might still be hard to understand after those sections, which is why there will also be a usually brief explanation of the logic that was used to reach the first output from the first input.

Table of contents

2D PROBLEMS (2D)	4
2D1: Embedded squares (15 points)	4
2D2: Library (35 points)	6
2D3: Big noise (75 points)	9
DATA SORTING (DS)	12
DS1: The perfect Gift! (30 points)	12
DS2: Data Packet Relay (35 points)	14
DS3: Simple linked list (15 points)	16
VIDEO GAMES (VG)	18
VG1: Overcooked (30 points)	18
VG2: Collision (35 points)	21
VG3: Pac Man (40 points)	23
VG4: Tetris (45 points)	26
TEXT PROCESSING (TP)	29
TP1: Columns and rows (40 points)	29
TP2: Haiku (20 points)	31
TP3: Palindroming (55 points)	33
TP4: Pangram (35 points)	35

A diagram showing a large rectangle with a smaller rectangle inside it. The large rectangle has a solid black border. The smaller rectangle inside it also has a solid black border. The large rectangle is labeled with a '1' on its left side and a '2' on its right side. The smaller rectangle is labeled with a '3' on its left side and a '4' on its right side.

```

wall = [
"      ",
"      ",
"      ",
"      ",
"      ",
"      "
]

```

```

wall = [
"      ",
"      ",
"      ",
"      ",
"      ",
"      ",
"      ",
"      ",
"      "
]

```

Sample output:

```

254
152
166

```

First output explanation:

In the first example, we have a wall of 22 characters wide which represents a real width of 22 feet. For the height, we have 7 characters which corresponds to a real height of 14 feet. This gives us a total surface including the window of $22 \times 14 = 308$ square feet.

The window has a width of 9 characters that represents 9 feet. The height of the window is 3 characters that represents a real height of 6 feet. The area of the window is $9 \times 6 = 54$ square feet.

We can subtract the 2 areas to obtain a final area of $308 - 54 = 254$

2D2: Library (35 points)

You want to start an online library and want to have a fancy display for your books that are on the shelves. To make it nice, you decide to arrange the books from the tallest on the left, to the smallest on the right.

The height of each book is determined by the length of its title since it will be displayed on the binding of the book. For books of the same height, they will need to be displayed in alphabetical order.

To make your book bindings, use this as the top part: " `┌───┐` ", the bottom part of the book is the following: " `└───┘` " and, for the section with letters used: " `| X |` " In that format, you have the side, a space then the letter, another space and finally the other side.

Input Specification:

You will receive:

- **books:** an *array of string* that are the names of the books.

Output Specification:

You will need to output:

- an *array of string* that are the visual representation of the bookshelf.

Sample input:

```
["Hobbit", "Disney", "Inferno", "Narnia", "Wicked", "Harry  
Potter"]
```

```
["The Hunger Games", "The Great Gatsby", "The book thief", "The  
Outsiders", "The little Prince", "Pride and Prejudice"]
```

```
["1984", "Twilight", "The Giver", "Divergent", "Holes", "The  
Host", "Uglies", "Eragon"]
```

Sample output:

H a r r y					
P o t t e r	I n f e r n o	D i s n e y	H o b b i t	N a r n i a	W i c k e d

P r i d e					
a n d	T h e	T h e	T h e	T h e	T h e
P r e j u d i c e	l i t t l e	G r e a t	H u n g e r	b o o k	O u t s i d e r s
	P r i n c e	G a t s b y	G a m e s	t h i e f	

D i v e r g e n t	T h e	T h e	T w i l i g h t				
	G i v e r	H o s t		E r a g o n	U g l i e s	H o l e s	1 9 8 4

First output explanation:

In the first example, we start by organizing books from the longest title to the shortest. Once they are in height order, we need to reorganize books to make sure that the ones with the same height are in alphabetical order. We then obtain the following list:

Harry Potter

Inferno

Disney

Hobbit

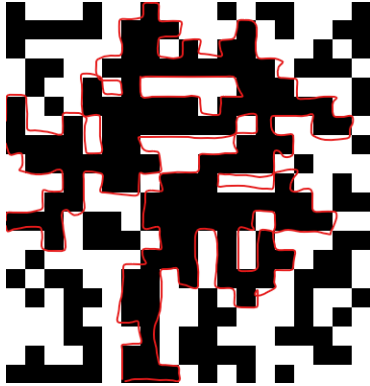
Narnia

Wicked

The only thing left is to display the bookshelf with the book bindings!

2D3: Big noise (75 points)

You must calculate the perimeter of the largest noise block (which is the block with the longest perimeter) on a given grid. A noise block is a set of noise pixels connected orthogonally (no diagonals, pixels must share a side to be connected). For example:



Don't forget to compute the perimeter of internal regions if applicable.

Input specification:

For input, you will receive:

- an *array* of *strings* with “.” to represent empty squares and with “#” to represent the squares that are full.

Input specification:

For output, you will return:

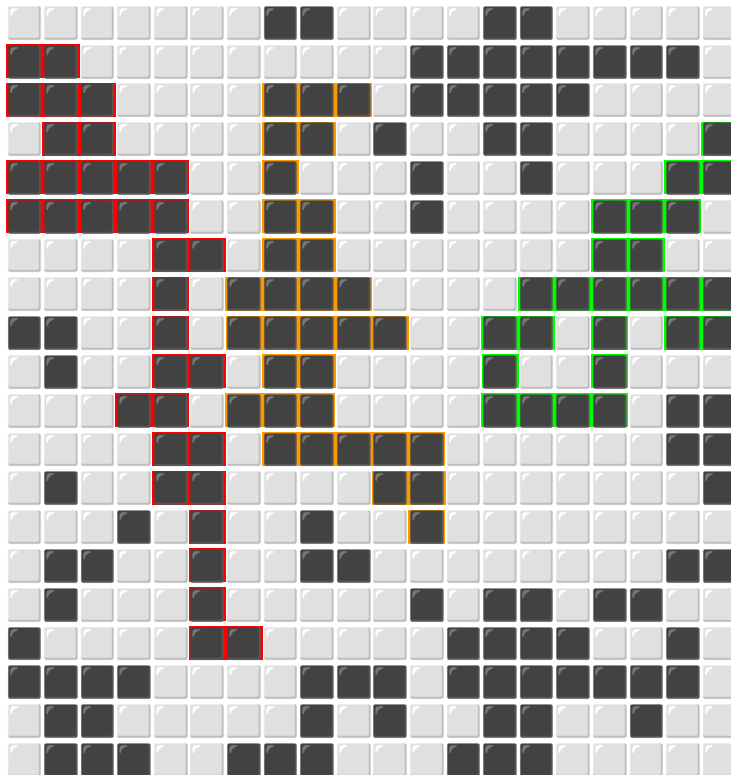
- an *int* that is the perimeter of the biggest noise block (internal + external)

Sample input:

```
[ ".....##....##.....",
  "##.....#####.",
  "###....###.#####.",
  ".##....##.#.##....#",
  "#####..#...#.#...##",
  "#####..##..#....###.",
  "...##.##.....##...",
  "...#.#####....#####",
  "##..#.#####..##.##.",
  ".#..##.##....#.#...",
  "...##.###....#####.",
  "...##.#####.....##.",
  ".#..##....##.....#.",
  "...#.##..#.#.....",
  ".##..#..##.....##.",
  ".#...#.....#.#.##.",
  "#....##.....#####.",
  "#####....###.#####.",
  ".##.....#.#.##..#..." ]
```

54
84
104

First output explanation:



In order of longest to shortest perimeters, we have:

Red block: 54, Orange block: 46, Green block: 44

The other blocks are no way near those numbers.

DATA SORTING (DS)

DS1: The perfect Gift! (30 points)

Santa Claus needed some help for the next year for the gift distribution system. For every child, Santa wants to find what would be the best gift to offer each child taking into account their interest and their age.

Every Toy is comprised of the following characteristics:

- **id** : the toy ID (string)
- **minAge** : the minimum recommended age (inclusively, int)
- **maxAge** : the maximum recommended age (inclusively, int)
- **category** : the category of the toy (string)
- **price** : the price to build the toy (int)

Selection rules:

1. The age given needs to be within the minAge and maxAge inclusively
2. The category needs to be right one

All valid toys need to be ordered ascendingly starting from the lowest price to the highest price and to be returned in an *array* of the toys' IDs.

Input Specification:

You will receive:

- **toys**: an *array* of *ToyInfo* of the different toys with their characteristics
- **age**: an *int* of the age of the child
- **category**: a *string* of the category that the child is interested in

Output Specification:

You will need to output:

- an *array* containing the *string* of the ID of all the valid toys ordered by increasing price

Sample input:

```
toys = [
    ToyInfo(3, 6, Category.PUZZLE, 15, 101, "Animal Puzzle"),
    ToyInfo(5, 9, Category.FIGURINE, 20, 102, "Knight Figure"),
    ToyInfo(6, 12, Category.PUZZLE, 10, 103, "World Map Puzzle"),
    ToyInfo(4, 7, Category.CAR, 18, 104, "Race Car"),]
age = 6
category = Category.PUZZLE

toys = [
    ToyInfo(7, 12, Category.BOARD_GAME, 35, 201, "Treasure Island"),
    ToyInfo(6, 10, Category.BOARD_GAME, 25, 202, "Math Challenge"),
    ToyInfo(8, 14, Category.COLLECTIBLE, 40, 203, "Rare Cards"),
    ToyInfo(3, 7, Category.BOARD_GAME, 30, 204, "Family Fun"),]
age = 8
category = Category.BOARD_GAME

toys = [
    ToyInfo(10, 16, Category.VIDEO_GAME, 60, 301, "Space
Adventure"),
    ToyInfo(8, 12, Category.VIDEO_GAME, 45, 302, "Puzzle Quest"),
    ToyInfo(12, 18, Category.VIDEO_GAME, 70, 303, "Battle Arena"),
    ToyInfo(9, 14, Category.CAR, 40, 304, "Remote Racer"),]
age = 11
category = Category.VIDEO_GAME
```

Sample output:

[103, 101]

[202, 201]

[302, 301]

First output explanation:

In the first example, we only keep Animal Puzzle and World Map Puzzle as they are the only 2 toys provided that are puzzles (the required category). We then look at age restrictions and, in that case, both toys are recommended for 6 year olds.

We can now order them by price. This gives us World Map Puzzle as the cheapest (ID: 103) followed by Animal Puzzle as the most expensive (ID:101).

DS2: Data Packet Relay (35 points)

A network consists of N machines (numbered 1 through N), each running its own local server. Every machine has a single outbound packet queued up: machine i intends to send its packet to machine a_i , where $a_i \neq i$.

A scheduling order is defined by a permutation $(p_1, p_2, \dots, p_N)$ of $1 \dots N$. The transmissions are then carried out sequentially as follows.

For each i from 1 up to N :

- If the intended recipient, machine a_{p_i} , has already shut down its server (because it previously sent its own packet), then machine p_i 's transmission is dropped and it stays idle.
- Otherwise, machine p_i successfully delivers its packet to machine a_{p_i} 's server, transferring v_{p_i} bytes of payload data. Immediately after sending, machine p_i shuts down its own server.

Determine the maximum amount of payload (in bytes) that can be successfully delivered across all machines, considering every possible scheduling order p .

Hint:

You may assume that there are at least 2 machines in the network, and that no machine is trying to send a packet to itself (i.e., $a_i \neq i$ for all i).

A permutation of $1 \dots N$ means a rearrangement of the numbers 1 through N .

For example, let $N = 3$, then $(1, 2, 3)$, $(2, 3, 1)$ and $(3, 1, 2)$ are some of the valid permutations, while $(1, 2, 2)$ and $(4, 1, 3)$ are not.

Input Specification:

You will receive:

- N , an *integer*, which denotes the number of machines in the network.
- a , an *array of N integers*, where a_i denotes the target machine that machine i 's packet is destined for.
- v , an *array of N integers*, where v_i denotes the payload size in bytes that machine i will deliver upon a successful transmission.

Output Specification:

You will need to output:

- a single *integer*, which denotes the maximum amount of payload (in bytes) that can be delivered across all machines, over every possible scheduling order p .

Sample input:

n = 4

a = [2, 4, 1, 3]

v = [20, 60, 40, 80]

n = 2

a = [2, 1]

v = [100, 50]

n = 5

a = [2, 1, 4, 5, 3]

v = [10, 30, 20, 40, 15]

Sample output:

180

100

90

First output explanation:

Consider the scheduling order $p = (3, 1, 2, 4)$:

- Machine 3 sends its packet to machine 1's server, delivering 40 bytes.
- Machine 1 sends its packet to machine 2's server, delivering 20 bytes.
- Machine 2 sends its packet to machine 4's server, delivering 60 bytes.
- Machine 4 attempts to reach machine 3, but machine 3 has already shut down.
Transmission is dropped.

The total here is $40 + 20 + 60 = 120$ bytes.

Now consider the scheduling order $p = (2, 4, 3, 1)$:

- Machine 2 sends its packet to machine 4's server, delivering 60 bytes.
- Machine 4 sends its packet to machine 3's server, delivering 80 bytes.
- Machine 3 sends its packet to machine 1's server, delivering 40 bytes.
- Machine 1 attempts to reach machine 2, but machine 2 has already shut down.
Transmission is dropped.

The total here is $60 + 80 + 40 = 180$ bytes. It can be proven that no scheduling order achieves a higher total than this.

DS3: Simple linked list (15 points)

Linked lists are really useful in programming yet they receive very little attention. They can be really simple and they have a sizable advantage over *arrays* because they don't need a fixed size so it is really easy to add new elements and keep the order without having to move every element.

Linked lists can be really simple by containing 2 pieces of information for each element in the list. The first piece is the information that you want to store and the second is the pointer to the next element of the list.

Having pointers to the next element makes it possible for the next information to be stored anywhere on a computer without any problem. You only have to follow the links to get the information from the linked list! We will give you an array containing elements of a linked list. From a given starting point, you will have to reconstruct the sequence of the elements in the linked list. The last element of the linked list will be the one that points towards an invalid position (-1).

Input Specification:

You will receive:

- **memory:** an *array* containing the elements of the linked list (the information will be int)
- **start:** an *int* of the starting position in the *array* of the start of the linked list

Output Specification:

You will need to output:

- an *array* of the information (int) from the linked list in order

Sample input:

```
[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]
start: 0
```

```
[(5, -1), (2, 2), (58, 0), (61, 1), (77, -1)]
start: 3
```

```
[(49, 2), (90, 0), (1, -1), (44, 6), (26, 4)]
start: 1
```

Sample output:

```
[17, 1, 14, 22]
```

```
[61, 2, 58, 5]
```

```
[90, 49, 1]
```


First output explanation:

In the first example, we have the following elements in memory:

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Since the start of our linked list is at position 0, the first element is 17. The pointer of that memory cell is toward cell 2.

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Here we have the value of 1 to add to our elements and the pointer toward the cell 1.

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Here we have the value 14 and we are pointing at cell 3.

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

We then have the value 22 and the pointer that points towards -1. Since -1 is not a valid position, we have reached the end of the linked list. This gives us the following elements that we stored:

[17, 1, 14, 22]

VIDEO GAMES (VG)

VG1: Overcooked (30 points)

You are playing *Overcooked* with your best friend and really struggling on a particularly difficult level — so much so that you begin to wonder whether the level is even theoretically possible. You therefore decide to calculate the maximum profit that can be achieved in this level and, to do so, you choose to write a program to compute it.

That being said, you will need to start relatively simple. For this reason, we will **not** consider the distances between the different tasks in the kitchen or the time required to move between them (otherwise this would become a rather complex optimization problem!). A list of ingredients corresponding to the recipe to be prepared will be given as input, and you must sell as many completed dishes as possible within the time limit (remember that you have two players who can perform tasks simultaneously!) in order to maximize your profits.

Each ingredient in the recipe may be preceded by **one (and only one)** adjective indicating how it must be prepared. The possible adjectives are:

- **<none>** meaning the ingredient is not preceded by any adjective and consists of a single word. An unprepared ingredient requires **1 second** of a player's time to add to the recipe.
- **"cut"**: the ingredient must be chopped on a cutting board. Chopping the ingredient and adding it to the recipe requires **5 seconds** of a player's time. There are enough cutting boards available for both players to chop at the same time.
- **"boiled"**: the ingredient must be boiled in a pot. An ingredient takes **10 seconds** to boil but only requires **1 second** of a player's time to add to the recipe. In other words, a pot is occupied for 10 seconds, but those 10 seconds do not count toward player time. You have **4 pots** that can boil independently in your kitchen.
- **"cooked"**: the ingredient must be cooked in a pan. An ingredient takes **7 seconds** to cook but only requires **1 second** of a player's time to add to the recipe. In other words, a pan is occupied for 7 seconds, but those 7 seconds do not count toward player time. You have **2 pans** that can cook independently in your kitchen.

In addition to all the ingredients, you must subtract an additional **1 second** of one player's time per dish to pick up the plate on which the dish will be served.

For this problem, we assume that any task started by one player can be finished by the other, and that all required working seconds can be distributed arbitrarily between the two players. In other words, for the purposes of this problem, **1 player with 120 seconds is virtually indistinguishable from 2 players with 60 seconds**. We also assume that, similarly to *Overcooked*, the order in which tasks are completed does not matter.

All that remains is to calculate the profit generated per dish according to the preparation level of each ingredient and the total number of ingredients using the following formula:

$$profit = (n_{total\ ingredients} + n_{not\ prepared} + 2n_{cut} + 4(n_{cooked} + n_{boiled})) * \frac{n_{total\ ingredients}}{3}$$

Input Specification:

You will receive:

- **timer**: an *int* of the number of seconds in the simulated level
- **recipe**: an *array* of *strings* that are the ingredients and the cooking instructions for the recipe

Output Specification:

You will need to output:

- **profit**: an integer corresponding to the total profit generated by selling recipes in the simulated level. If you obtained a float value for the total level profit, it is your responsibility to round it to the nearest whole number and convert it to an integer.

Sample input:

```
240, ["bread", "cut lettuce", "cut tomato", "cooked beef"]
300, ["cut carrot", "cut cabbage"]
265, ["pepper", "cut cheese", "cut tomato", "boiled pasta",
"boiled sauce", "cooked beef"]
150, ["cooked sausage", "cooked bread"]
407, ["boiled rice", "boiled carrot", "cooked beef", "cooked
onion"]
```

Sample output:

```
624
216
1610
140
1547
```

First output explanation:

This is a burger recipe!

We must begin by considering the cooking process and by determining how many beef patties we can cook in the pans. Each pan can cook

$240/7=34.28... \Rightarrow 34$ beef patties. With two pans working at the same time, that gives us a maximum of 68 patties.

As for the players, the recipe requires:

1 second to pick up the plate,

1 additional second to place the bun on it,

10 seconds to chop the lettuce and the tomato,

1 final second to add the cooked beef.

This gives a total of 13 seconds per recipe.

Using 240 seconds for 2 players, we have a total of 480 player-seconds available. Therefore, the players can assemble at most

$480/13=36.92...$

$480/13=36.92... \Rightarrow 36$ recipes within the allotted time.

Since we have enough cooked ingredients for all these recipes, this becomes our final count.

Using the profit formula, we obtain:

$(4+1+2 \times 2+4 \times 1) \times 4/3=17.33...$

$(4+1+2 \times 2+4 \times 1) \times 4/3=17.33...$ profit per burger,

for a total of

$36 \times 17.33...=624$

$36 \times 17.33...=624$

profits that can be generated in this level.

VG2: Collision (35 points)

In the world of video games, a fundamental element handled by game engines is collision detection between two objects. Often, objects in a game are made up of cubes, spheres, or other basic solids.

In this problem, you must determine whether two objects are colliding or not. You will be given a sphere and a cube, and you must decide whether or not they intersect. The cube will always be oriented parallel to all axes. In other words, no rotation is applied to the cube. If it were on the ground, it would sit flat. For the sphere, there is also no rotation to consider for collisions, because it is a sphere...

For both the sphere and the cube, you will receive the 3D position of the object's center followed by its dimension. For the sphere, the given dimension will be the radius. For the cube, the given dimension will be the length of one of its sides.

A small geometric reminder: for the sphere, the volume includes everything inside the radius (inclusive) from the center of the object. For the cube, starting from a corner, everything within one side length along each of the three axes is included. Be careful — this logic starts from a corner, but you will be given the center of the cube as input!

A brief reminder of 3D distance geometry:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

Input Specification:

You will receive:

- **sphere:** a *tuple* of 4 *int* that are the coordinates in 3D of the center and then the radius
- **cube:** a *tuple* of 4 *int* that are the coordinates in 3D of the center and then the side length

Output Specification:

You will need to output:

- a *bool* of if there is a collision between the sphere and cube

Sample input:

sphere = (4, 5, 6, 2)

cube = (6, 6, 6, 2)

sphere = (11, 14, 3, 3)

cube = (9, 12, 11, 6)

```
sphere = (0, 0, 0, 10)
cube = (0, 9, 14, 10)
```

Sample output:

True

False

True

First output explanation:

In the first example, we have a cube with its center at position $x = 6$, $y = 6$, $z = 6$ and a side length of 2. Since it is a cube with side length 2 and we are given the center, we move 1 unit in each direction along every axis to find its corners.

We obtain the cube's 8 corners as:

[5, 5, 5], [5, 5, 7], [5, 7, 5], [7, 5, 5], [5, 7, 7], [7, 5, 7], [7, 7, 5], [7, 7, 7]

One of the corners closest to the center of the sphere is the point (5, 5, 5). We can calculate the distance between this point and the sphere's center (4, 5, 6) and check whether it is smaller than the sphere's radius to determine if there is a collision.

$$\begin{aligned}d &= \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \\d &= \sqrt{(4 - 5)^2 + (5 - 5)^2 + (6 - 5)^2} \\d &= \sqrt{(-1)^2 + 0^2 + 1^2} \\d &= \sqrt{2}\end{aligned}$$

Since the square root of 2 is smaller than 2, we have a collision between the cube and the sphere. The answer is True, we do have a collision.

VG3: Pac Man (40 points)

One of the most famous video games of all time is the iconic game **Pac-Man**. The principle of the game is relatively simple. The yellow Pac-Man must move around and collect as many points as possible on a game board while avoiding the ghosts that are chasing him.

You must determine which ghost is the closest to catching Pac-Man on a given grid. To calculate the distance, you must use the shortest path between each ghost and Pac-Man. The ghosts move orthogonally (up, down, left, right), and each square traveled to reach Pac-Man counts as a distance of 1.

In this problem, walls are represented by “#”, valid movement spaces are represented by “.”, Pac-Man is represented by the letter P and, finally, the ghosts are represented by the numbers from 1 to 4, corresponding to the four ghosts chasing Pac-Man.

Input Specification:

You will receive:

- **grid:** an *array of string* of the current state of the game.

Output Specification:

You will need to output:

- an *int* of the number of the closest ghost

Sample input:

```
"#####",
"#.....#.....#",
"#.####.####.#.####.####.#",
"#.####.####.#.####.####.#",
"#.....1.....#",
"#.####.#.#####.#.####.#",
"#.....#.....#.....#.....#",
"#####.####.#.####.####.",
"#####.#.P.....#.#####",
"#####.#.####.####.#.#####",
"#.....##.....##.....#",
"#####.#.#####.#.#####",
"#####.#.....4#.#####",
"#####.#.#####.#.#####",
"#..3.....#.....#",
"#.####.#.####.####.#.####.#",
"#....#.....#.....#",
"####.#.#.#####.#.#.####",
"#.....#.....#.....#.....#",
"#.#####.#.#####.#",
"#.....2....#",
"#####"
```

```
"#####",
"#..4.....#.....#",
"#.####.####.#.####.####.#",
"#.####.####.#.####.####.#",
"#.....#.....#",
"#.####.#.#####.#.####.#",
"#.....#.....#.....#1.....#",
"#####.####.#.####.####.",
"#####.#....P.....#.#####",
"#####.#.####.####.#.#####",
"#.....##.....##.....#",
"#####.#.#####.#.#####",
"#####.#....2.....#.#####",
"#####.#.#####.#.#####",
"#.....#.....3#",
"#.####.#.####.####.#.####.#",
"#....#.....#.....#",
"####.#.#.#####.#.#.####",
"#.....#.....#.....#.....#",
"#.#####.#.#####.#",
"#.....#.....#",
"#####"
```


Sample output:

3

2

First output explanation:

We compute the distance of the shortest path between each ghost and Pac-Man and we choose the one with the smallest distance.

```
"#####",
"#.....#",
"#.####.#.####.####.",
"#.####.#.####.####.",
"#.....1.....#",
"#.####.#.#####.#.####.",
"#.....#.....#.....#",
"#####.#.#####.",
"#####.#P.....#.",
"#####.#.####.#.####.",
"#.....#.#.....#.....#",
"#####.#.#####.#.####.",
"#####.#.#####.#.####.",
"#####.#.#####.#.####.",
"#####.#.#####.#.####.",
"#####.#.#####.#.####.",
"#.3.....#.....#",
"#.####.#.####.#.####.",
"#.....#.....#.....#",
"#####.#.#####.#.####.",
"#.....#.....#.....#.....#",
"#.#####.#.#####.#.",
"#.....#.....#.....#.....#",
"#.#####.#.#####.#.",
"#####"
```

The distance between ghost 1 and Pac-Man is 15 squares.
The distance between ghost 1 and Pac-Man is 30 squares.
The distance between ghost 1 and Pac-Man is 12 squares.
The distance between ghost 1 and Pac-Man is 13 squares.
Since ghost 3 is the closest, we return 3.

VG4: Tetris (45 points)

The goal of Tetris is to stack shapes on top of one another while trying to complete full rows in order to clear them and reduce the height of the stack. There are many strategies to optimize one's score, but they all rely on knowing where the piece will land. Fortunately for players, most Tetris games display the final landing position of the piece. Your task is to recreate this feature.

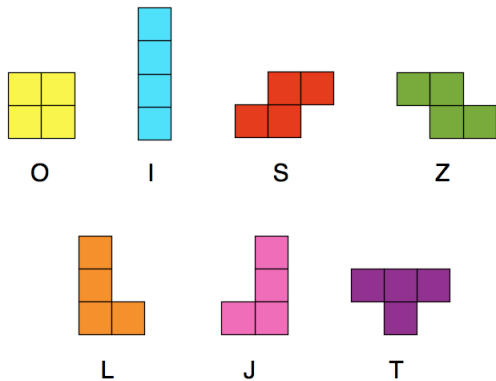
Recall that a piece stops moving as soon as:

- it touches the bottom of the board, or
- it collides with an existing block.

Input Specification:

You will receive:

- Board: a 10×8 *list* filled with 0s and 1s representing the already occupied cells.
- Piece: an uppercase letter representing the piece to place in the orientation shown below.
- Position: an *int* between 1 and 10 representing the leftmost column (from left to right) where the game piece appears.



Output Specification:

You will need to output:

- Board: the same *list* given as input, but now including the placed piece, with its squares marked using "x".

Sample input:

```
1. Board = [
    [0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0],
```

```

[1,0,0,0,1,0,1,1,1,1],
[1,1,1,0,1,1,1,1,1,1]]
Piece = 'I'
Position = 6
2. Board = [
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,1,1,0,0,0],
[1,1,0,0,0,1,1,1,1,1],
[1,1,1,0,1,1,1,1,1,1]]
Piece = 'T'
Position = 4
3. Board = [
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[1,1,1,0,0,0,0,0,0,0],
[1,1,1,1,0,0,1,0,0,1],
[1,1,1,1,1,1,1,0,0,1]]
Piece = 'O'
Position = 8
4. Board = [
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[1,0,0,0,0,0,0,0,0,1],
[1,1,0,0,1,1,0,0,1,1],
[1,1,1,0,1,1,1,1,1,1]]
Piece = 'S'
Position = 3

```

Sample output:

```

1. Board = [
[0,0,0,0,0,0, 0 ,0,0,0,0],

```

```

[0,0,0,0,0, 0 ,0,0,0,0],
[0,0,0,0,0, 0 ,0,0,0,0],
[0,0,0,0,0,"x",0,0,0,0],
[0,0,0,0,0,"x",0,0,0,0],
[0,0,0,0,0,"x",0,0,0,0],
[1,0,0,0,1,"x",1,1,1,1],
[1,1,1,0,1, 1 ,1,1,1,1]]

2. Board = [
[0,0,0, 0 , 0 , 0 ,0,0,0,0],
[0,0,0, 0 , 0 , 0 ,0,0,0,0],
[0,0,0, 0 , 0 , 0 ,0,0,0,0],
[0,0,0, 0 , 0 , 0 ,0,0,0,0],
[0,0,0,"x","x","x",0,0,0,0],
[0,0,0, 0 ,"x", 1 ,1,0,0,0],
[1,1,0, 0 , 0 , 1 ,1,1,1,1],
[1,1,1, 0 , 1 , 1 ,1,1,1,1]]

3. Board = [
[0,0,0,0,0,0,0, 0 , 0 ,0],
[0,0,0,0,0,0,0, 0 , 0 ,0],
[0,0,0,0,0,0,0, 0 , 0 ,0],
[0,0,0,0,0,0,0, 0 , 0 ,0],
[0,0,0,0,0,0,0, 0 , 0 ,0],
[1,1,1,0,0,0,0, 0 , 0 ,0],
[1,1,1,1,0,0,1,"x","x",1],
[1,1,1,1,1,1,1,"x","x",1]]

4. Board = [
[0,0, 0 , 0 , 0 ,0,0,0,0,0],
[0,0, 0 , 0 , 0 ,0,0,0,0,0],
[0,0, 0 , 0 , 0 ,0,0,0,0,0],
[0,0, 0 , 0 , 0 ,0,0,0,0,0],
[0,0, 0 , 0 , 0 ,0,0,0,0,0],
[1,0, 0 ,"x","x",0,0,0,0,1],
[1,1,"x","x", 1 ,1,0,0,1,1],
[1,1, 1 , 0 , 1 ,1,1,1,1,1]]

```

First output explanation:

First, the ‘T’ piece must be placed correctly in its horizontal orientation. Position 4 means that the leftmost block of the ‘T’ must be in column 4. Once the piece is aligned, determine which column causes the piece to collide first as it “falls,” and place the piece at that height.

TEXT PROCESSING (TP)

TP1: Columns and rows (40 points)

A cypher method that was used during the first and even second world war is a column cypher. How it works is that words are set up in a grid and instead of being read from left to right, they are read from top to bottom.

So once you receive a text, it has to be arranged in columns, and then it can be read as the original message. The challenging part of the solution is to figure out how many columns are being used to decipher the secret message. In our case, the amount of rows and columns is going to be the 2 closest factors (that multiply to the amount of characters in the special encrypted message. So the grid would be as close as possible to be a square. The grid can be the size of the number of characters in the message (including spaces and punctuation) or can even be the message with an empty space added at the end of it if it helps to make a more square grid.

When there is a different amount of columns than rows, There will always be more columns than rows. (The grids will always the product of 2 numbers or 1 less than the product of 2 numbers)

For 48 characters, it can be divided as 6x8 but even better, it can be referred to as 7x7 with one unused letter. So the message needs to be decrypted as a 7x7 grid since an added empty character makes the grid have the same amount of columns and rows.

Here's a few more examples:

- For a message of 30 characters, it can be divided as 3x10 or 6x5 so we take 6x5 since it is the closer amount of rows to columns ratio. We organize it into 6 columns and 5 rows.
- For a message of 29 characters it would also be 6x5 but with one empty space at the end of the grid.
- For a message of 24 characters, it is 6x4 but can also be represented as 5x5-1. We then have to take 5x5 as the grid size.

Input Specification:

You will receive:

- **text**: a *string* of the encrypted message.

Output Specification:

You will need to output:

- a *string* that is the decrypted message

Sample input:

C wc xeyaalfspryao shessrpie s rnraaitogsl cec

Blssrreatya g pvmedtoe éé sec!

A pgaha eicrny!

Sample output:

Cyphers are always a classic for text processing

Bravo le message est décrypté!

Again a cypher!

First output explanation:

In the first example we have the following encrypted message:

C wcxeyeaalftspryao shessrpie s rnraaitorgsl cec

That message contains a total of 48 characters that can be divided in 6x8. But, if we consider the fact that we can have an empty space at the end of the message, we can have the number of rows and columns be both 7. Therefore, we use a grid that is 7x7 for the message decryption which gives us the following grid:

C		w	c		x	e
y	a	a	l	f	t	s
p	r	y	a	o		s
h	e	s	s	r	p	i
e			s		r	n
r	a	a	i	t	o	g
s	l		c	e	c	

We can then extract the message by reading the columns instead of the rows and have "Cyphers are always a classic for the text processing" as the final output!

TP2: Haiku (20 points)

Haikus are Japanese poems that are divided into three lines based on the number of syllables. For the purposes of this problem, haikus will instead be divided based on the number of letters.

A word cannot be split unless it contains punctuation such as an apostrophe or a hyphen.

To form a haiku in this problem, the first line must contain 10 letters.

The second line must contain 7 letters.

And the third and final line must contain 13 letters.

Input Specification:

As input you will receive:

- **haiku:** a *string* of the message to try to format as a haiku

Output Specification:

As output you will need to provide:

- ONE *string* that is either “Impossible” if it cannot be formatted as a haiku and the formatted haiku otherwise. The line changes needs to be indicated with \n and all the unnecessary spaces needs to be removed

Sample input:

At the CRC we never give up and lose the game!

Un bon robot ne gagne que facilement!

It is hard to write on limited length.

Un arc-en-ciel est magnifique et impressionnant

Sample output:

Impossible

Un bon robot
ne gagne
que facilement!

It is hard to
write on
limited length.

Impossible

First output explanation:

In the first sentence, the words “At the CRC we” have 10 letters, so the first line is correct! Then “never give” has 9 letters, not 7, so it's impossible to write a haiku in the given format.

TP3: Palindroming (55 points)

You love palindromes, that is, a word, number, phrase, or other sequence of symbols that reads the same backwards as forwards, such as "madam" or "racecar". You find them fascinating, and you enjoy creating them.

You have a computer that can help you process palindromes. However, it takes a huge amount of resources to run, so you want to minimize the number of executions.

Your task is simple: given a random string s , and you want to transform it into a palindrome. You can either remove characters from the string, which will cost you 1 unit amount of resources per character removed, or you can simply swap the characters in the string. If you want to swap two characters that are d distance apart, it will cost you d units of resources (i.e., swapping characters at index 2 and index 5 costs 3 units).

Your goal is to find the minimum amount of resources needed to transform the string s into a palindrome.

Input Specification:

You will receive:

- a *string*, solely consisting of lowercase ASCII characters (a-z), as the only input.

Output Specification:

You will need to output:

- a single *integer*, representing the minimum amount of resources needed to transform the string into a palindrome.

Sample input:

crc

mrgoosrem

amanbaplnaacqsanalpwamana

Sample output:

0

3

First output explanation:

Note that the string "crc" is already a palindrome, so no resources are needed to transform it.

Second output explanation:

In the string "mrgeosrem", we can first drop the character g at index 2 and the character e at index 7, which will cost us 2 units of resources and give "mroosrm". Then, we can swap the characters o at index 3 and s at index 4, which will cost us 1 unit of resources and give "mrosorm". Which is a palindrome. The total cost is $2 + 1 = 3$ units of resources.

TP4: Pangram (35 points)

Pangrams are a type of phrase that include every letter of the alphabet. The most impressive ones are the ones that contain as few repetitions as possible. You will have to make a program that validates if the sentence contains every letter of the alphabet. If it contains every letter of the alphabet, your program will need to output all the duplicate letters in order of appearance.

Input Specification:

You will receive:

- a *string* of the sentence to study to validate if it is a pangram.

Output Specification:

You will need to output:

- a *string* that is empty if the sentence is not a pangram and if it is a pangram, it should contain the letters in the order of appearance.

Sample input:

The quick brown fox jumps over the lazy dog

Sphinx of black quartz, judge my vow

aaaaaabcdefghijklmnopqrstuvwxyz

Portez ce vieux whisky au juge blond qui fume

Sample output:

"ouoerteo"

"auo"

""

"eeiueouiue"

First output explanation:

In the sentence "The quick brown fox jumps over the lazy dog" which is the most known pangram, we have all the letters from the alphabet included. In that case, we need to extract all the doubles in order. In that case, most of the doubles are vowels except for r and t. Notice that for the output only the doubles are selected and in order of appearance of the repetition.